

Valve Inventory

Upgrade Guide

How to move from an older installed version to a newer one - say, v1.3 to v1.4 - without losing anything you've added. This is a living document: the general procedure below applies to every release so far and is expected to keep applying, but check "Version-specific notes" at the end for anything a particular release calls out as different.

The short answer

No, it isn't export-and-reimport. All of your data - stock, box locations, confirmed parameters, everything - lives in one file, `valves.db`, and upgrading never touches that file directly. Back it up (below), get the new version's code, and run it against your existing database. That's the whole procedure.

Note: This works because every schema change so far has been strictly additive - new tables and new columns only, never a renamed or removed one. The app brings its schema up to date against your existing database on every single startup (`V.init_db()`): it creates whatever tables are new (`CREATE TABLE IF NOT EXISTS`) and adds whatever columns are new (`ALTER TABLE ... ADD COLUMN`, from v1.4 on). On an up-to-date database that's a no-op; on an older one it happens in place, leaving every value already there untouched - SQLite's `ADD COLUMN` appends a nullable column to the table definition without rewriting any existing row.

1. Back up first - always

Do this before touching anything else. Two options, and it's fine to do both:

Copy the file (fastest, most robust)

Copy `valves.db` itself to somewhere safe - another folder, a dated backup folder, cloud storage, a USB stick. If you've built a local datasheet archive, copy the `datasheets/` folder too (optional - it's rebuildable, just slower to redo than to copy).

Refresh the text snapshot

```
python3 snapshot.py
```

Writes a human-readable copy into `data/` - good to have regardless, and if you track your own fork in git, commit it so you have real version history of your collection, not just a single backup snapshot.

2. Get the new version

If you're on a git clone

```
git fetch --tags
```

```
git checkout v1.4          # or: git pull, to track main instead of a tag
```

`valves.db` is gitignored, so neither command touches it - only the code and docs update. Nothing further to do for the database itself.

If you're using a downloaded copy (no git)

Download the new release and extract it to a **new** folder - don't extract over the old one. Then copy your existing `valves.db` (and `datasheets/`, if you have one, and anything under `docs/screenshots/` you added yourself) from the old folder into the new one.

3. Run it

```
python3 valves_gui.py
```

First launch against the upgraded code creates any new tables the new version needs, against your existing data, automatically. You should see your full collection immediately - same counts, same boxes, same confirmed parameters.

4. Confirm nothing's missing

```
python3 test_smoke.py
```

Then Tools > Collection summary (or `python3 valves.py stats`) and check the totals match what you expect. If anything looks off, your backup from step 1 is right there.

Optionally, refresh the snapshot again now that you're upgraded, so `data/` reflects the current version's schema too:

```
python3 snapshot.py
```

What not to do

Note: Don't run `snapshot.py --restore` as part of upgrading. That rebuilds `valves.db` FROM the last-committed `data/valves.sql` - which discards anything you've added to your live database since the last time you ran plain `snapshot.py`. It's the right tool for a fresh clone that has no database yet at all (see the Installation Manual), not for upgrading one you already have.

Version-specific notes

Nothing beyond the standard procedure above has ever been required. Entries below record what each release actually changed about the database, and are worth a read before upgrading - but if there's nothing here for the version you're moving to, the standard procedure is all you need.

v1.4

Adds six optional columns to the `stock` table - `position`, `type1`, `type2`, `origin`, `test_values` and `other` - so a lot can record where in its box it sits, what else the valve is marked as, and where it came from. This is the first release to add columns to an existing table rather than whole new tables, so it is the first to run an `ALTER TABLE` against your database. It still needs no manual steps: first launch adds the columns in place and leaves every existing value untouched, and the new columns start out empty on every lot you already have. Fill them in as and when it's worth it - blank is a perfectly normal value, and nothing else in the tool changes behaviour because of them.

It also adds two new tables, `valve` (one row per individually-tracked physical valve) and `valve_test` (one row per test of one), created automatically on first run like any other new table. Nothing about your existing collection changes when they appear: every lot stays exactly as it was, held as a quantity, until you expand one. See "Individual valves and testing" in the User Manual.

Note: One behaviour change worth knowing about even if you never expand a lot. **Take** now removes individual valve records alongside the quantity, and deleting a lot removes its valves and their tests. On a collection with nothing expanded there is nothing to remove and Take behaves exactly as before - but once you have recorded tests, a Take is the one thing that can discard them. It picks the least documented valves first precisely to make that unlikely.

Note: Two things worth knowing if you script against the database yourself. The `v_stock` view is dropped and recreated on that first launch - it gains `position`, the alternative designations, `origin` and an `individuals` count - so any view of your own built *on top of* `v_stock` should be checked afterwards. And `data/stock.csv` gains six columns (plus two new files, `valves.csv` and `tests.csv`), so the first `snapshot.py` run after upgrading will show a large diff for that file even if you've changed nothing - that's the header and the new empty fields, not lost data.

v1.3

Adds one new table (`document` - extra datasheets, links, and the general reference library) and a Docs tab. Purely additive, created automatically on first run - no manual steps.

v1.1 - v1.2

v1.1 added an auto-restore prompt for a brand-new database with no `valves.db` yet; irrelevant if you're upgrading an existing one. No schema changes. (There was no separate v1.2 release.)